

PetCare API

Especificación Funcional de la API (Contrato)

Versión 1.2 · Contrato funcional corregido y congelado para implementación

Propósito del documento Definir qué debe hacer la primera versión de PetCare API, qué datos intercambia, quién puede ejecutar cada operación y qué comportamiento debe considerarse parte del contrato. Swagger/OpenAPI será la representación interactiva de este contrato, no su reemplazo.

Proyecto	PetCare API
Tipo	API REST para libreta sanitaria digital de mascotas
Versión contractual	1.2
Estado	Aprobado como idea por el docente · pendiente de implementación
Fecha	09/09/2026

Control del documento

Versión	Fecha	Estado	Cambio
1.0	09/09/2026	Baseline	Definición funcional inicial de PetCare API V1.
1.1	09/09/2026	Revisión documental	Separación de reglas de negocio y diccionario de errores hacia el Documento 02.
1.2	09/09/2026	Baseline corregido	Se cierran contratos de refresh/logout, PATCH nullable, fechas de vacunación y respuestas exitosas.

Cómo debe leerse este documento

La propuesta de “Especificación Funcional de la API (Contrato)” se mantiene como documento principal para definir qué ofrece PetCare, cómo se consume y qué datos intercambia. OpenAPI/Swagger UI será su representación interactiva. Las reglas de negocio detalladas y el catálogo de errores se documentan por separado para evitar duplicación y mantener responsabilidades claras.

- Swagger/OpenAPI documentará y permitirá probar el contrato HTTP de manera interactiva.
- El contrato de datos definirá campos, tipos, obligatoriedad, validaciones y significado de negocio.
- Este documento concentra alcance, actores, recursos, permisos, endpoints, contratos de datos, seguridad y criterios de aceptación.
- Se evitan IDs mágicos para estados o roles: el contrato usa enums semánticos como OWNER, CAREGIVER y VIEWER.

Decisión de alcance Esta especificación describe exclusivamente la V1 necesaria para el Integrador 4. Todo lo marcado como “futuro” queda explícitamente fuera del compromiso de implementación.

Índice funcional

Secciones 1–8	Secciones 9–15
1. Objetivo y audiencia	2. Problema, propuesta de valor y alcance
3. Actores y modelo de acceso	4. Modelo funcional de datos
5. Reglas de negocio — referencia	6. Convenciones generales de la API

Secciones 1–8	Secciones 9–15
7. Autenticación y sesión	8. Catálogo de endpoints
9. Contrato de datos	10. Errores — referencia
11. Seguridad funcional	12. OpenAPI / Swagger UI
13. Criterios de aceptación V1	14. Fuera de alcance y evolución
15. Alineación con Integrador 4	

1. Objetivo y audiencia

PetCare API provee una capa de servidor segura y persistente para gestionar mascotas compartidas entre una o más personas responsables y registrar su historial básico de vacunación. La API está diseñada como backend independiente; el frontend queda fuera del alcance de esta versión.

La audiencia principal de este documento es:

- Product Owner / docente: para comprender qué resuelve el producto, qué recursos expone y qué comportamiento contractual queda comprometido.
- Desarrollo backend: como fuente de verdad funcional antes de implementar controllers, services, DTOs y persistencia.
- QA / evaluación: para derivar casos de prueba y verificar respuestas esperadas.
- Integración futura: para que un frontend o consumidor externo conozca rutas, datos y reglas sin leer el código fuente.

2. Problema, propuesta de valor y alcance

2.1 Problema que resuelve

La información básica de una mascota suele quedar distribuida entre libretas físicas, mensajes, fotografías o recuerdos de distintas personas. Esto se vuelve especialmente incómodo cuando la mascota vive en una familia, está al cuidado temporal de otra persona o proviene de una organización de rescate.

2.2 Propuesta de valor V1

PetCare centraliza la identidad básica de la mascota, las personas que tienen acceso y los registros de vacunación. La V1 no intenta reemplazar a una veterinaria ni emitir recomendaciones clínicas: organiza información aportada por los usuarios.

2.3 Alcance comprometido

- Registro, login, refresh y logout de usuarios con JWT.
- Creación y CRUD de mascotas.
- Acceso compartido a una mascota mediante roles simples.
- CRUD de vacunaciones asociadas a una mascota.
- Persistencia en PostgreSQL mediante Prisma.
- Protección de rutas, validación de DTOs y seguridad HTTP básica.
- Documentación interactiva mediante OpenAPI / Swagger UI.
- Deploy funcional de la API y de la base de datos.

2.4 Principio de diseño del MVP

Regla del MVP No se incorpora una funcionalidad nueva si no es necesaria para cumplir el Integrador o para sostener correctamente el modelo de acceso compartido. La complejidad se concentra en modelado, autenticación, autorización y seguridad.

3. Actores y modelo de acceso

3.1 Actor: Usuario autenticado

Persona con una cuenta PetCare válida. Un mismo usuario puede tener distintos roles en distintas mascotas.

3.2 Roles sobre una mascota

Rol	Descripción de negocio	Permisos V1
OWNER	Responsable administrador de la mascota. Puede haber más de uno.	Lectura, edición, vacunaciones, gestión de accesos y eliminación.
CAREGIVER	Persona que participa activamente en el cuidado.	Lectura, edición de datos básicos y CRUD de vacunaciones. No administra accesos ni elimina la mascota.
VIEWER	Persona con acceso de consulta.	Solo lectura de mascota y vacunaciones.

3.3 Conceptos humanos que NO son roles técnicos

“Pareja”, “familiar”, “persona de respaldo” o “voluntario de ONG” describen la relación real entre personas, pero no definen permisos. La API traduce esas situaciones a OWNER, CAREGIVER o VIEWER según el nivel de acceso requerido.

3.4 Matriz de autorización

Operación	OWNER	CAREGIVER	VIEWER
Ver mascota	Sí	Sí	Sí
Ver vacunaciones	Sí	Sí	Sí
Crear vacunación	Sí	Sí	No
Editar vacunación	Sí	Sí	No
Eliminar vacunación	Sí	Sí	No
Editar mascota	Sí	Sí	No
Agregar acceso	Sí	No	No
Cambiar rol	Sí	No	No
Quitar acceso	Sí	No	No
Eliminar mascota	Sí	No	No

4. Modelo funcional de datos

User N — N Pet
mediante PetAccess

Pet 1 — N Vaccination

4.1 User

Representa la cuenta de acceso. Sus credenciales pertenecen al subsistema de autenticación y nunca deben exponerse en las respuestas.

4.2 Pet

Representa una mascota. No contiene un único ownerId porque la V1 admite múltiples responsables. La relación con usuarios se resuelve a través de PetAccess.

4.3 PetAccess

Entidad intermedia explícita entre User y Pet. Guarda el rol del usuario en esa mascota. La combinación userId + petId debe ser única.

4.4 Vaccination

Registro de una vacuna aplicada. Se limita deliberadamente a información declarativa: nombre de la vacuna, fecha de aplicación, próxima fecha sugerida si fue informada, datos opcionales del profesional/centro y notas.

4.5 Situación de cuidado y rescate

Pet.careMode describe la situación de cuidado actual con los valores FAMILY o FOSTER. rescueOrganizationName es un dato descriptivo opcional. En V1 una ONG no es una entidad autenticable ni administra miembros; si una persona vinculada a una ONG necesita acceso, utiliza una cuenta User normal.

5. Reglas de negocio — referencia contractual

Las reglas de negocio detalladas no se duplican en este contrato. Se mantienen en el Documento 02 — Diccionario de Errores y Reglas de Negocio de PetCare V1. Este documento solo conserva su impacto contractual: qué rol puede ejecutar cada endpoint, qué campos son obligatorios y qué respuestas HTTP puede producir cada operación.

Reglas especialmente relevantes para integración: crear una mascota asigna al creador como OWNER; una mascota puede tener varios responsables; la gestión de accesos requiere OWNER; y el sistema no permite dejar una mascota sin al menos un OWNER. El detalle, identificadores BR-* y escenarios límite se especifican en el Documento 02.

6. Convenciones generales de la API

Elemento	Contrato V1
Base path	/api/v1
Formato	JSON UTF-8
IDs	UUID generados por Prisma/base de datos
Fechas de negocio	YYYY-MM-DD. En PostgreSQL se modelan como date; no representan hora.
Timestamps técnicos	createdAt / updatedAt en ISO 8601.
Nombres de propiedades	camelCase
Enums	Strings semánticos; no IDs numéricos mágicos.
Actualización parcial	PATCH
Autenticación	Authorization: Bearer <accessToken> en rutas protegidas.
Content-Type	application/json

6.1 Convención de listas

La V1 devuelve listas sin paginación porque el objetivo académico y el volumen esperado son acotados. Todas las colecciones se encuentran limitadas por autorización al usuario o mascota correspondiente. La paginación queda como evolución futura.

6.2 Convención de respuestas

Los endpoints exitosos devuelven directamente el recurso o colección correspondiente, evitando envelopes innecesarios. Los errores siguen una estructura consistente de statusCode, error y message.

7. Autenticación y sesión

7.1 Flujo

```

register → UserPublic
login → accessToken + refreshToken + user
accessToken → autoriza rutas privadas (~15 min)
refreshToken → POST /auth/refresh body → nuevo accessToken
logout → POST /auth/logout body → invalida refreshTokenHash

```

access token ya emitido puede seguir válido hasta expirar

7.2 Tokens

Token	Duración objetivo	Uso	Persistencia
Access token	~15 minutos	Acceso a rutas protegidas.	No se persiste como texto plano en DB.
Refresh token	~7 días	Renovar access token.	Se conserva únicamente su hash para poder invalidarlo.

7.3 Contrato de refresh y logout

POST /api/v1/auth/refresh y POST /api/v1/auth/logout reciben { "refreshToken": "<jwt>" } en JSON. El backend valida firma, expiración y correspondencia con refreshTokenHash. V1 mantiene un único refresh activo por usuario y no implementa rotación: /refresh devuelve solo un accessToken nuevo.

Logout invalida la capacidad de renovar la sesión al dejar refreshTokenHash en null. Como el access token es stateless, un access token emitido previamente puede seguir siendo aceptado hasta su expiración corta; el cliente debe descartarlo al cerrar sesión.

7.4 Publicidad de rutas

La API se diseña como privada por defecto. Health, register, login y Swagger son públicas. Refresh y logout no requieren access token, pero sí un refreshToken válido en el body; por eso se excluyen del JwtAuthGuard de access y realizan su propia validación de sesión. El resto de rutas requiere un access token válido.

8. Catálogo de endpoints

8.1 Health

Método	Ruta	Auth	Descripción
GET	/api/v1/health	Pública	Confirma que la API responde; útil para deploy/smoke test.

8.2 Auth

Método	Ruta	Auth	Éxito	Descripción
POST	/api/v1/auth/register	Pública	201	Crea una cuenta y devuelve UserPublic.
POST	/api/v1/auth/login	Pública	200	Valida credenciales y devuelve AuthResponse: access + refresh + user.
POST	/api/v1/auth/refresh	refreshToken en body	200	Valida el refresh vigente y devuelve solo un nuevo accessToken; no rota refresh en V1.
POST	/api/v1/auth/logout	refreshToken en body	204	Valida el refresh vigente, pone refreshTokenHash en null y responde sin body.
GET	/api/v1/auth/me	Access token	200	Devuelve el perfil público del usuario actual.

8.3 Pets — CRUD principal #1

Método	Ruta	Permiso	Éxito	Descripción
POST	/api/v1/pets	Usuario autenticado	201	Crea mascota y asigna al creador como OWNER.
GET	/api/v1/pets	Cualquier PetAccess	200	Lista las mascotas accesibles para el usuario.

Método	Ruta	Permiso	Éxito	Descripción
GET	/api/v1/pets/:petId	Cualquier PetAccess	200	Obtiene una mascota.
PATCH	/api/v1/pets/:petId	OWNER / CAREGIVER	200	Actualiza parcialmente datos básicos.
DELETE	/api/v1/pets/:petId	OWNER	204	Elimina mascota y relaciones dependientes.

8.4 Accesos compartidos

Método	Ruta	Permiso	Éxito	Descripción
GET	/api/v1/pets/:petId/access	OWNER	200	Lista usuarios y roles de la mascota.
POST	/api/v1/pets/:petId/access	OWNER	201	Agrega un usuario existente por email.
PATCH	/api/v1/pets/:petId/access/:userId	OWNER	200	Cambia el rol de acceso.
DELETE	/api/v1/pets/:petId/access/:userId	OWNER	204	Quita un acceso si no deja a la mascota sin OWNER.

8.5 Vaccinations — CRUD principal #2

Método	Ruta	Permiso	Éxito	Descripción
POST	/api/v1/pets/:petId/vaccinations	OWNER / CAREGIVER	201	Registra una vacunación.
GET	/api/v1/pets/:petId/vaccinations	OWNER / CAREGIVER / VIEWER	200	Lista vacunaciones de la mascota.
GET	/api/v1/pets/:petId/vaccinations/:vaccinationId	OWNER / CAREGIVER / VIEWER	200	Obtiene una vacunación.
PATCH	/api/v1/pets/:petId/vaccinations/:vaccinationId	OWNER / CAREGIVER	200	Actualiza parcialmente la vacunación.

Método	Ruta	Permiso	Éxito	Descripción
DELETE	/api/v1/pets/:petId/ vaccinations/:vaccinationId	OWNER / CAREGIVER	204	Elimina una vacunación.

Fuera del contrato V1 Filtros de “próximas” o “vencidas” pueden añadirse después de completar y desplegar el core. No son criterio de aceptación de esta primera versión.

9. Contrato de datos

9.1 RegisterRequest

Campo	Tipo	Req.	Reglas	Significado
name	string	Sí	2–80 caracteres; trim	Nombre visible del usuario.
email	string	Sí	Formato email; lowercase; único	Identificador de inicio de sesión.
password	string	Sí	8–72 caracteres	Se recibe en texto solo durante la request y se hashea antes de persistir.

```
{
  "name": "Maga",
  "email": "maga@example.com",
  "password": "*****"
}
```

9.2 LoginRequest

Campo	Tipo	Req.	Reglas	Significado
email	string	Sí	Email válido	Cuenta a autenticar.
password	string	Sí	No vacío	Credencial a verificar con bcrypt.

9.3 AuthResponse

Campo	Tipo	Req.	Significado
accessToken	string	Sí	JWT corto para rutas protegidas. Login lo entrega; refresh devuelve uno nuevo.
refreshToken	string	Sí	JWT largo de sesión. Login lo entrega; V1 no lo rota en /refresh.
user	UserPublic	Sí	Datos públicos del usuario autenticado. Presente en login.

9.3.1 Contratos de sesión complementarios

RegisterResponse = UserPublic. LoginResponse = AuthResponse { accessToken, refreshToken, user }.
 RefreshRequest = { refreshToken }; RefreshResponse = { accessToken }. LogoutRequest = { refreshToken };
 LogoutResponse = 204 No Content, sin body. HealthResponse = { "status": "ok" }.

9.4 UserPublic

Campo	Tipo	Req.	Significado
id	uuid	Sí	Identificador público.
name	string	Sí	Nombre visible.
email	string	Sí	Email de la cuenta.
createdAt	datetime	Sí	Fecha/hora técnica de creación.

Dato sensible passwordHash y refreshTokenHash jamás forman parte de UserPublic ni de ninguna respuesta HTTP.

9.4.1 Semántica general de PATCH

En todo PATCH de PetCare: un campo omitido conserva el valor persistido; un campo nullable enviado como null elimina su valor; un valor no nulo reemplaza el anterior. Las reglas que relacionan varios campos se validan sobre el estado final resultante de combinar el registro existente con el DTO recibido.

9.5 CreatePetRequest / UpdatePetRequest

Campo	Tipo	Create	Update	Reglas / negocio
name	string	Req.	Opc.	2-80 caracteres.
species	enum	Req.	Opc.	DOG CAT OTHER.
breed	string null	Opc.	Opc.	Máx. 80 caracteres. PATCH: omitido conserva; null elimina el valor; valor reemplaza.
birthDate	date null	Opc.	Opc.	YYYY-MM-DD; no futura. PATCH: omitido conserva; null elimina el valor; valor reemplaza.
careMode	enum	Req.	Opc.	FAMILY FOSTER.
rescueOrganizationName	string null	Opc.	Opc.	Dato descriptivo; máx. 120 caracteres. PATCH: omitido conserva; null elimina el valor; valor reemplaza.

9.6 PetResponse

Campo	Tipo	Significado
id	uuid	Identificador de la mascota.
name	string	Nombre.
species	enum	Especie normalizada.
breed	string null	Raza declarada.
birthDate	date null	Fecha de nacimiento, si se conoce.
careMode	enum	Situación actual de cuidado.
rescueOrganizationName	string null	ONG de referencia, si corresponde.
myRole	enum	Rol del usuario actual para esta mascota.
createdAt	datetime	Alta técnica.

Campo	Tipo	Significado
updatedAt	datetime	Última modificación.

```
{
  "id": "a8f1...",
  "name": "Luna",
  "species": "CAT",
  "breed": null,
  "birthDate": "2024-03-10",
  "careMode": "FOSTER",
  "rescueOrganizationName": "Patitas Felices",
  "myRole": "OWNER",
  "createdAt": "2026-09-09T18:10:00.000Z",
  "updatedAt": "2026-09-09T18:10:00.000Z"
}
```

9.7 CreatePetAccessRequest

Campo	Tipo	Req.	Reglas / negocio
email	string	Sí	Debe corresponder a un usuario existente y no asociado ya a la mascota.
role	enum	Sí	OWNER CAREGIVER VIEWER.

9.8 UpdatePetAccessRequest

Campo	Tipo	Req.	Reglas / negocio
role	enum	Sí	OWNER CAREGIVER VIEWER. No puede degradar al último OWNER.

9.9 PetAccessResponse

Campo	Tipo	Significado
userId	uuid	Usuario con acceso.
name	string	Nombre público del usuario.
email	string	Email para identificar al colaborador.
role	enum	Rol sobre esa mascota.

Campo	Tipo	Significado
createdAt	datetime	Fecha de asignación del acceso.

9.10 CreateVaccinationRequest / UpdateVaccinationRequest

Regla de consistencia: appliedAt debe ser <= a la fecha actual. Si nextDueAt existe, debe ser >= appliedAt. En PATCH, ambas reglas se evalúan sobre el estado final combinado, no solamente sobre los campos presentes en el body.

Campo	Tipo	Create	Update	Reglas / negocio
vaccineName	string	Req.	Opc.	2–120 caracteres.
appliedAt	date	Req.	Opc.	YYYY-MM-DD; no puede ser futura.
nextDueAt	date null	Opc.	Opc.	Si existe, debe ser >= appliedAt. PATCH: omitido conserva; null elimina; valor reemplaza.
veterinarianName	string null	Opc.	Opc.	Máx. 120 caracteres. PATCH: omitido conserva; null elimina; valor reemplaza.
clinicName	string null	Opc.	Opc.	Máx. 120 caracteres. PATCH: omitido conserva; null elimina; valor reemplaza.
notes	string null	Opc.	Opc.	Máx. 1000 caracteres. PATCH: omitido conserva; null elimina; valor reemplaza.

9.11 VaccinationResponse

Campo	Tipo	Significado
id	uuid	Identificador del registro.
petId	uuid	Mascota a la que pertenece.
vaccineName	string	Nombre informado de la vacuna.
appliedAt	date	Fecha de aplicación.

Campo	Tipo	Significado
nextDueAt	date null	Próxima fecha informada, si existe.
veterinarianName	string null	Profesional informado.
clinicName	string null	Centro informado.
notes	string null	Notas libres.
createdAt	datetime	Alta técnica.
updatedAt	datetime	Última modificación.

10. Errores — referencia contractual

Cada endpoint documentará en Swagger las respuestas HTTP que pueden producirse (por ejemplo 400, 401, 403, 404 y 409), pero el catálogo de códigos de aplicación, mensajes y escenarios exactos vive en el Documento 02 — Diccionario de Errores y Reglas de Negocio.

El frontend no deberá inferir reglas de negocio a partir de textos libres. Cuando corresponda, la API expondrá un código de error estable junto con statusCode y message; el contrato detallado de esos códigos se define en el Documento 02.

11. Seguridad funcional

Control	Contrato V1
Password hashing	bcrypt con costo 10–12.
Access JWT	Token corto (~15 min).
Refresh JWT	Token largo (~7 días); hash persistido para invalidación.
AuthGuard	Rutas privadas protegidas; las públicas se declaran explícitamente.
ValidationPipe	Global con whitelist, forbidNonWhitelisted y transform.
Helmet	Cabeceras HTTP de seguridad.
CORS	Origen configurado explícitamente por variable de entorno.
Rate limiting	Global y reforzado en register/login.

Control	Contrato V1
Secrets	DATABASE_URL y secretos JWT solo por variables de entorno.
Logs	No registrar passwords, JWTs, refresh tokens ni hashes.
Respuestas	No exponer passwordHash, refreshTokenHash ni stack traces.

12. OpenAPI / Swagger UI

Swagger UI es la vista interactiva del contrato HTTP. No sustituye las reglas de negocio de este documento, pero debe mantenerse sincronizado con los DTOs y endpoints implementados.

Elemento	Definición V1
Swagger UI	GET /api/docs
OpenAPI JSON	GET /api/docs-json
Autenticación	Bearer JWT documentado mediante esquema de seguridad.
Agrupación	Tags: health, auth, pets, pet-access, vaccinations.
DTOs	Schemas generados/anotados para request y response.
Errores	Documentar por endpoint los status HTTP aplicables y referenciar el catálogo del Documento 02.

Swagger incluirá ejemplos mínimos de request/response para facilitar las pruebas del PO/docente.

12.1 Qué debe poder hacer un PO/docente desde Swagger

1. Registrar un usuario.
2. Iniciar sesión y copiar el access token.
3. Autorizar Swagger con Bearer token.
4. Crear una mascota.
5. Registrar una vacunación.
6. Consultar ambas entidades.
7. Probar una validación incorrecta y observar un 400.
8. Probar una operación sin autorización y observar 401/403 según corresponda.

13. Criterios de aceptación V1

ID	Criterio
AC-01	Un usuario puede registrarse con datos válidos y su password no queda en texto plano.
AC-02	Un usuario puede iniciar sesión y recibir access + refresh token.
AC-03	Refresh con refreshToken vigente genera un nuevo accessToken; logout con refreshToken vigente invalida refreshTokenHash. El access JWT previo puede vivir hasta expirar.
AC-04	Sin access token no se puede acceder a recursos privados.
AC-05	Crear una mascota crea también el acceso OWNER del creador de forma atómica.
AC-06	Un usuario ve únicamente mascotas para las que posee PetAccess.
AC-07	OWNER puede administrar accesos y no puede dejar una mascota sin OWNER.
AC-08	CAREGIVER puede mantener datos de mascota y vacunaciones, pero no accesos.
AC-09	VIEWER solo puede consultar.
AC-10	Pet CRUD funciona contra PostgreSQL real.
AC-11	Vaccination CRUD funciona contra PostgreSQL real, rechaza appliedAt futura, respeta nextDueAt >= appliedAt y valida el estado final en PATCH.
AC-12	DTOs inválidos y propiedades desconocidas son rechazados.
AC-13	La API aplica Helmet, CORS y rate limiting.
AC-14	Swagger permite descubrir y probar el contrato V1.

ID	Criterio
AC-15	La API y PostgreSQL están desplegados y mantienen persistencia después de un reinicio.

14. Fuera de alcance y evolución

Fuera de V1	Motivo
Organizaciones/ONG como entidad	Implicaría miembros, permisos y ownership organizacional.
Invitaciones por email	Requiere tokens, expiración, aceptación/rechazo y servicio de correo.
Adopciones / historial de custodia	Introduce un nuevo proceso de negocio.
Veterinarias / turnos	Nuevo dominio y recursos adicionales.
Medical records / diagnósticos	Dominio clínico fuera del objetivo.
Medicamentos / tratamientos	Amplía el alcance sanitario y reglas.
Fotos / documentos	Requiere almacenamiento de archivos.
Notificaciones / recordatorios	Requiere jobs y canales externos.
Frontend	Fuera del entregable del Integrador 4.
IA / RAG	No resuelve una necesidad del MVP.
Paginación avanzada	No necesaria para el volumen académico inicial.

15. Alineación con Integrador 4

El contrato V1 está deliberadamente ajustado al mínimo técnico solicitado para el Cuarto Proyecto Integrador. El foco no está en sumar features, sino en demostrar modelado relacional, autenticación, autorización, persistencia, validación y seguridad.

Requisito académico	Respuesta de PetCare V1
NestJS 11 + TypeScript	API modular con controllers, providers y DTOs.

Requisito académico	Respuesta de PetCare V1
Prisma + PostgreSQL	Persistencia y migraciones mediante schema.prisma.
Registro / login	Auth module con bcrypt.
Access + refresh JWT	Sesión completa con refresh invalidable.
Rutas protegidas	AuthGuard + autorización por PetAccess.
CRUD de al menos 2 recursos	Pet + Vaccination.
Validación global	class-validator / class-transformer + ValidationPipe.
Helmet / CORS / throttler	Controles transversales obligatorios.
README / documentación	README + Swagger/OpenAPI.
Deploy	API + PostgreSQL gestionado en proveedor.
GitHub	Repositorio nuevo con commits progresivos y Conventional Commits.

15.1 Evidencia mínima de finalización

- Repositorio público con código, schema.prisma, README.md y .env.example.
- Historial de commits progresivo y descriptivo.
- Build y migraciones exitosas.
- Deploy accesible.
- Swagger accesible y consistente con la implementación.
- Smoke test completo: register → login → create pet → create vaccination → read → refresh → logout con refreshToken.
- Al menos un test automatizado del flujo de autenticación.

Decisiones congeladas para comenzar implementación

- **Dominio V1: User, Pet, PetAccess, Vaccination.**
- Dos CRUD académicos: Pet y Vaccination.
- Relación User–Pet muchos-a-muchos mediante PetAccess explícito.
- Roles: OWNER, CAREGIVER, VIEWER.
- Situación de cuidado: FAMILY o FOSTER; ONG solo como metadata opcional.
- Sin invitaciones: el usuario a compartir debe existir previamente.
- Swagger forma parte de la entrega documental de V1.
- No se incorporan features futuras antes de completar, probar y desplegar este contrato.

Baseline Cualquier cambio que altere recursos, permisos, campos obligatorios o criterios AC-* debe actualizar este contrato. Las reglas BR-* y los códigos de error se mantienen en el Documento 02 y se sincronizan con este documento cuando afecten el contrato HTTP.

- Para un recurso existente sin PetAccess se mantiene 403 PET_ACCESS_DENIED como decisión consciente del MVP.
- Todos los OWNER tienen la misma autoridad, incluida la eliminación definitiva de la mascota; no existe PRIMARY_OWNER en V1.
- Vaccination: appliedAt no puede ser futura y nextDueAt, si existe, no puede ser anterior a appliedAt.
- PATCH: omitido conserva, null limpia campos nullable y las reglas cruzadas se validan sobre el estado final.
- Logout revoca la renovación de sesión; no se implementa blacklist de access JWT y el access emitido puede vivir hasta expirar.
- Auth V1: refresh y logout reciben refreshToken en JSON; no hay rotación y existe un solo refresh activo por usuario.

Base académica

Este contrato fue ajustado a la consigna “Cuarto Proyecto Integrador – Backend con NestJS” de IntegrarTEC (agosto 2026), que exige NestJS + TypeScript + Prisma + PostgreSQL, autenticación con access/refresh JWT, bcrypt, validación global, Helmet, CORS, rate limiting, al menos dos CRUD persistidos, documentación y deploy funcional.